

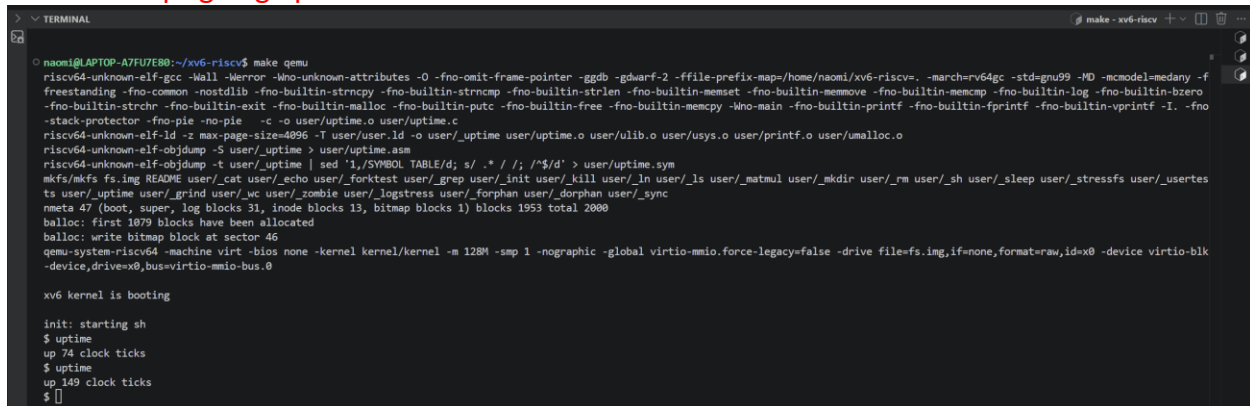
PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility

Your code `uptime.c` should be in the user directory in the `pa1` branch of your xv6 repo. Show results from running your `uptime` command.

I ran `uptime` two times, a few seconds apart. The first time it printed 74 ticks and the second time 149 ticks, so about 75 ticks (around 7.5 seconds) passed between them. This shows the counter keeps going up.



```
> ~ TERMINAL
@ make - xv6-riscv + v ...

naomi@LAPTOP-A7FUTE88:~/xv6-riscv$ make qemu
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -D -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/naomi/xv6-riscv- -march=rv64gc -std=gnu99 -MD -mmodel=medany -f
freestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero
-fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-fprintf -fno-builtin-vprintf -fno
-stack-protector -fno-pie -no-pie -c -o user/uptime.o user/uptime.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/uptime user/uptime.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/uptime > user/uptime.asm
riscv64-unknown-elf-objdump -t user/uptime | sed '1,/SYMBOL TABLE/d; s/ .* //; /$/d' > user/uptime.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_matmul user/_mkdir user/_rm user/_sh user/_sleep user/_stressfs user/_usertes
ts user/_uptime user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync
mmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1079 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk
-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ uptime
up 74 clock ticks
$ uptime
up 149 clock ticks
$
```

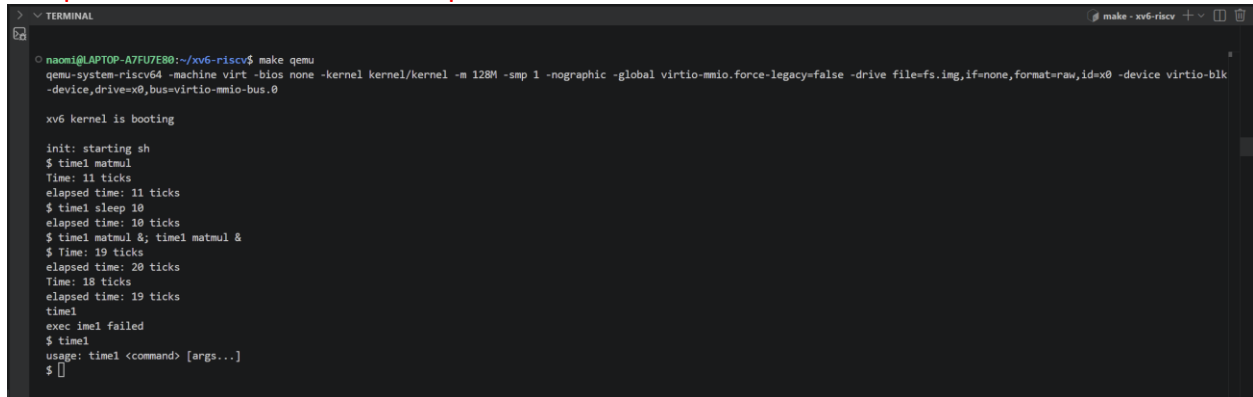
Task 2. (10 pts) Implement a `time1` command that reports elapsed time.

Your code `time1.c` should be in the user directory in the `pa1-syscalls` branch of your xv6 repo. Remember to add your new command to `UPROGS` in `Makefile`. Show results from running your `time1` command.

First it checks that the user actually typed a command after `time1`. If not (`argc < 2`), it prints a usage message and then exits. Then it saves the start time with `uptime()` and calls `fork()` to make a child process. `Fork()` returns 0 in the child and the child's PID in the parent, so I use that to make them do different things.

The child calls `exec()` to turn into the command. I pass `argv[1]` as the program name and `argv+1` as the argument list. If `exec()` comes back, it means it failed, so the child prints an error and exits.

The parent calls `wait(0)` to wait until the child is done, then saves the end time with `uptime()` and prints the end-start as the elapsed time.



```
naomi@LAPTOP-APFUE9B:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

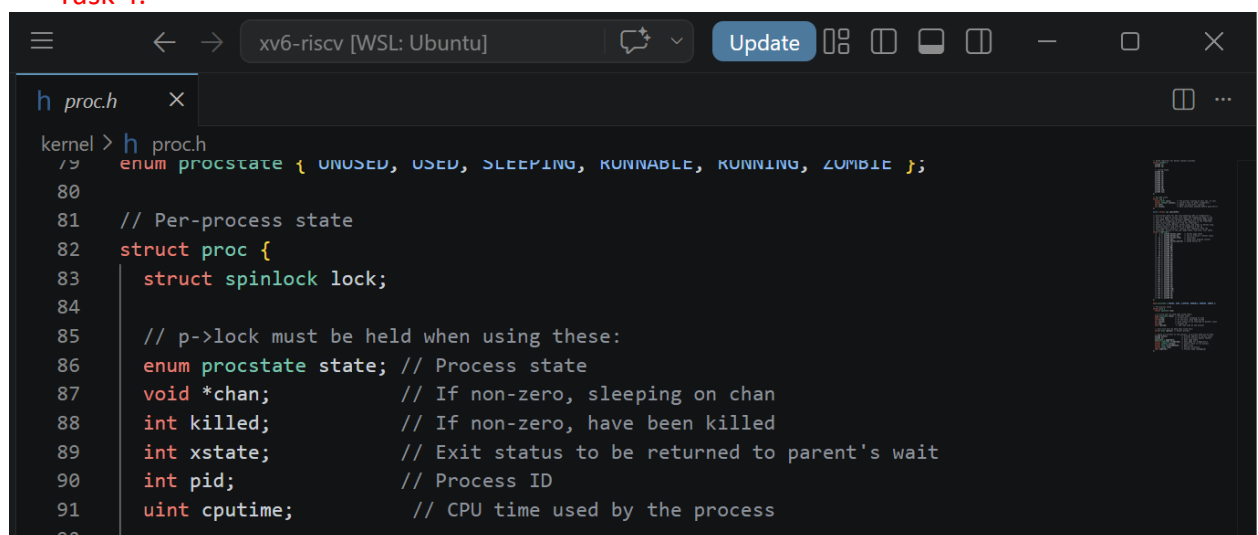
init: starting sh
$ time! matmul
Time: 11 ticks
elapsed time: 11 ticks
$ time! sleep 10
elapsed time: 10 ticks
$ time! matmul & time! matmul &
Time: 19 ticks
elapsed time: 20 ticks
Time: 18 ticks
elapsed time: 19 ticks
time!
exec imel failed
$ time!
usage: time! <command> [args...]
$
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

Describe what files you changed and how you changed them. Explain the purpose of each change. Be sure to do git add and git commit as needed and push your changes to your online repo.

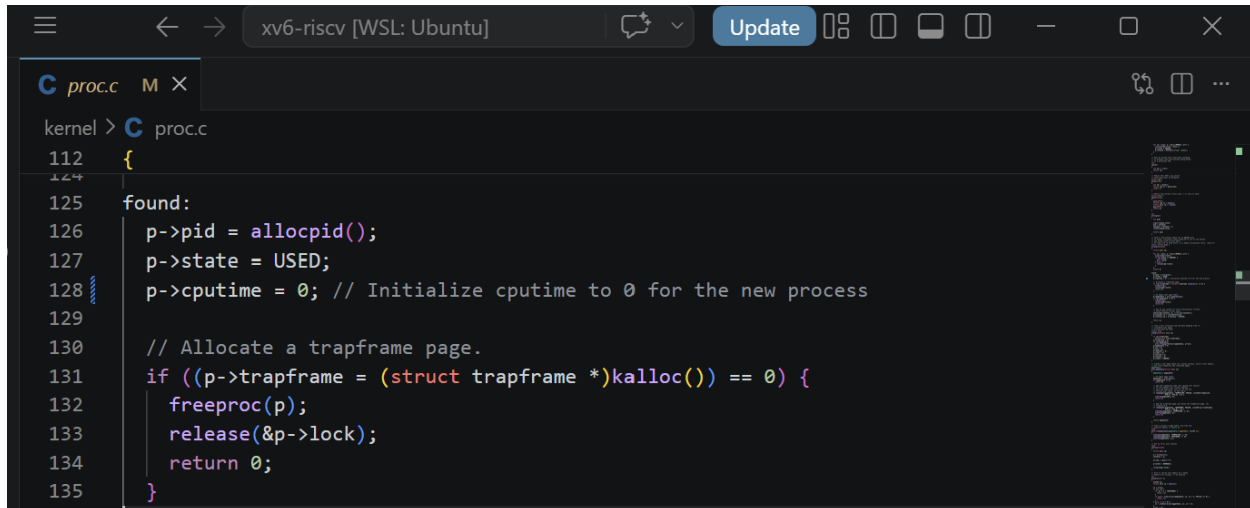
For this task I changed 3 files in the kernel. The idea is that every time the timer goes off (one tick), whatever process is running at that moment gets +1 added to its CPU time.

1. Kernel/`proc.h`: I added `uint cputime`, to `struct proc`. Struct `proc` is where the kernel keeps all the info about each process, so now every process has its own counter for how many ticks it has used. I used `uint` so it matches the `cputime` field in `struct rusage` from Task 4.



```
h proc.h x
kernel > h proc.h
19  enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
80
81  // Per-process state
82  struct proc {
83      struct spinlock lock;
84
85      // p->lock must be held when using these:
86      enum procstate state; // Process state
87      void *chan;           // If non-zero, sleeping on chan
88      int killed;           // If non-zero, have been killed
89      int xstate;           // Exit status to be returned to parent's wait
90      int pid;              // Process ID
91      uint cputime;          // CPU time used by the process
92
```

2. Kernel/proc.c: In allocproc(), right after the new process gets its PID, I added `p->cputime = 0`; `allocproc()` is where new processes get set up, so this makes sure every new process starts at 0 and doesn't keep an older number from a process that used the same slot before.



```
kernel > C proc.c
112 {
125 found:
126     p->pid = allocpid();
127     p->state = USED;
128     p->cputime = 0; // Initialize cputime to 0 for the new process
129
130     // Allocate a trapframe page.
131     if ((p->trapframe = (struct trapframe *)kalloc()) == 0) {
132         freeproc(p);
133         release(&p->lock);
134         return 0;
135     }
```

3. Kernel/trap.c: The timer interrupt can happen while the process is running its own code (usertrap) or while the kernel is doing something for it, like a system call (kerneltrap). In both places, when `which_dev == 2` (which means the interrupt came from the timer), I add 1 to the process's cputime right before `yield()` gives up the CPU. In `kerneltrap` I also check `myproc() != 0`, because sometimes the timer goes off when no process is running, and then there's nobody to charge. I also had to put the `{}` around two lines inside each `if`. Without them, only the first line belongs to the `if`, so `yield()` would run every time. Xv6 wouldn't even compile, because it treats warnings as errors.

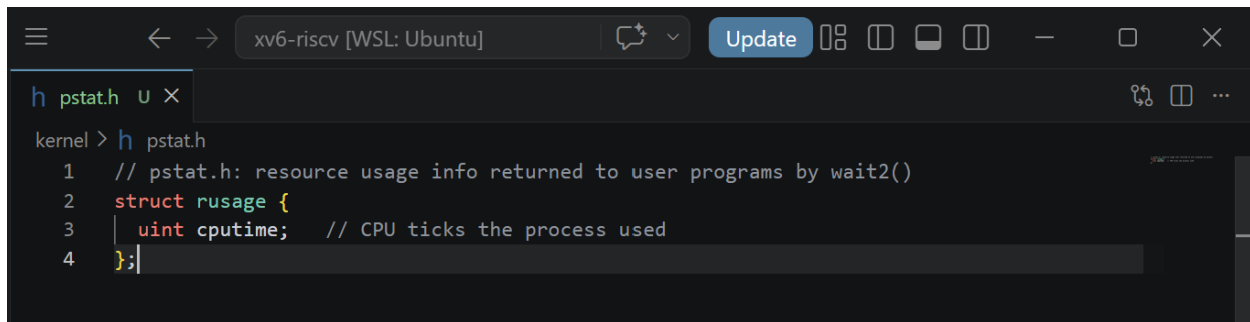
```
kernel > C trap.c
39 {
80
81     if (killed(p))
82         kexit(-1);
83
84     // give up the CPU if this is a timer interrupt.
85     if (which_dev == 2){
86         p->cputime += 1;
87         yield();
88     }
89
90     prepare_return();
91
92     // the user page table to switch to, for trampoline.S
93     uint64 satp = MAKE_SATP(p->pagetable);
94
95     // return to trampoline.S; satp value in a0.
96     return satp;
97 }
98

h proc.h
kernel > C trap.c
140 {
157
158     // give up the CPU if this is a timer interrupt.
159     if (which_dev == 2 && myproc() != 0){
160         myproc()->cputime += 1;
161         yield();
162     }
163
164     // the yield() may have caused some traps to occur,
165     // so restore trap registers for use by kernelvec.S's sepc instruction.
166     w_sepc(sepc);
167     w_sstatus(sstatus);
168 }
169
```

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

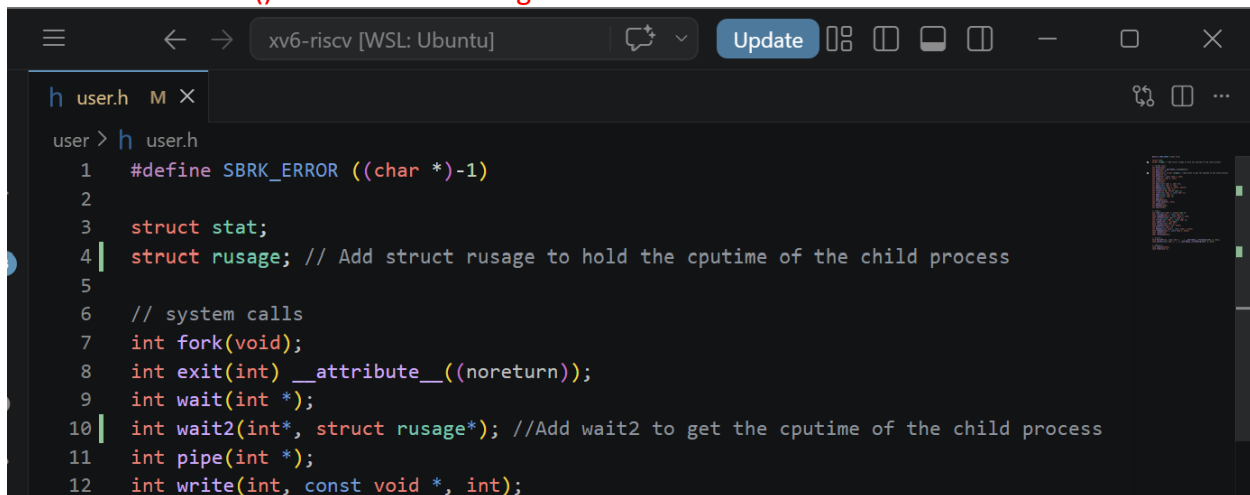
List the files you changed and explain the purpose of each change. Be sure to do git add and git commit as needed and push your changes to your online repo.

1. kernel/pstat.h (new file); defines struct rusage, which only has cputime for now. This is like the "package" the kernel fills in and sends back to the parent.



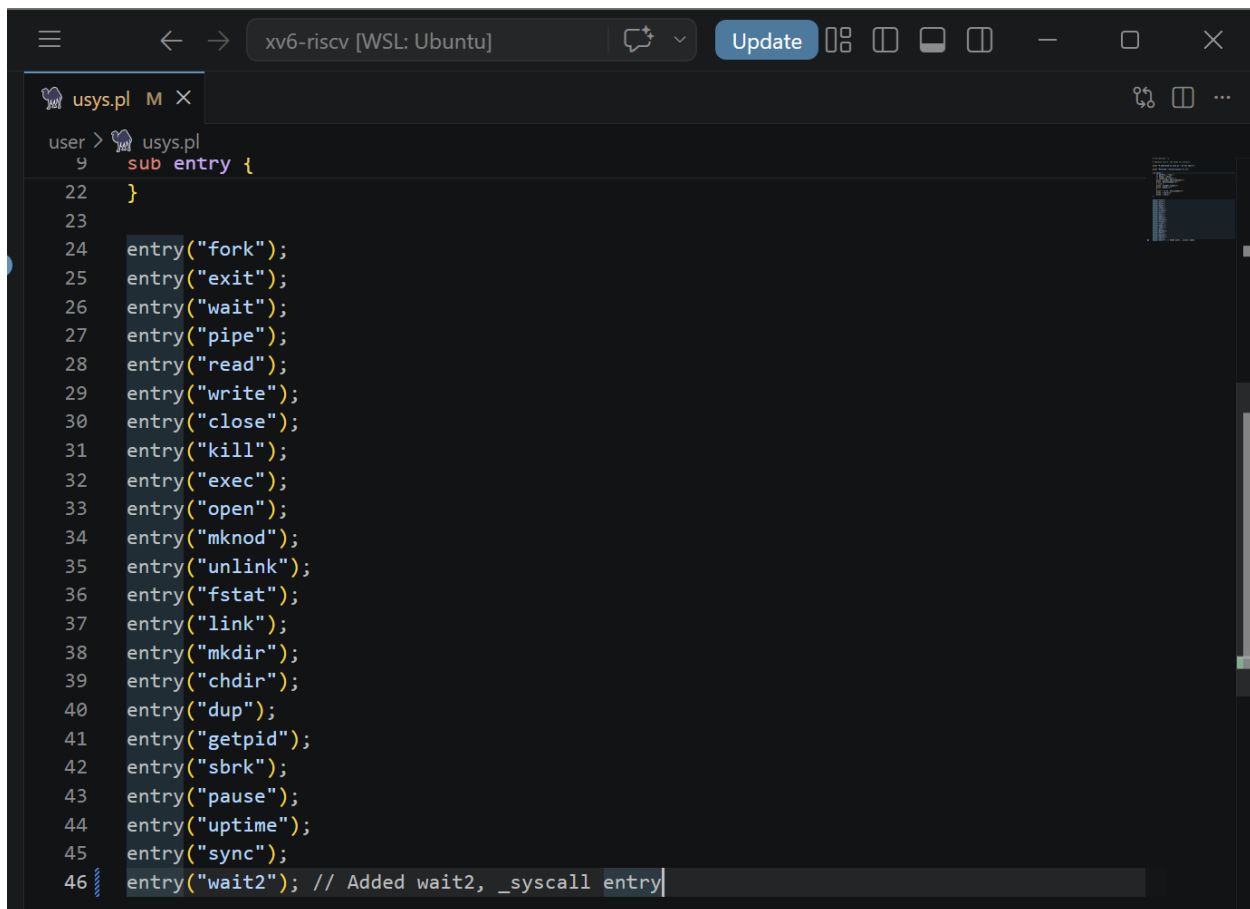
```
kernel > h pstat.h
1 // pstat.h: resource usage info returned to user programs by wait2()
2 struct rusage {
3     uint cputime; // CPU ticks the process used
4 };
```

2. user/user.h: Added struct rusage; and int wait2(int*, struct rusage*); so user programs know wait2() exists and what arguments it takes.



```
user > h user.h
1 #define SBRK_ERROR ((char *)-1)
2
3 struct stat;
4 struct rusage; // Add struct rusage to hold the cputime of the child process
5
6 // system calls
7 int fork(void);
8 int exit(int) __attribute__((noreturn));
9 int wait(int *);
10 int wait2(int*, struct rusage*); //Add wait2 to get the cputime of the child process
11 int pipe(int *);
12 int write(int, const void *, int);
```

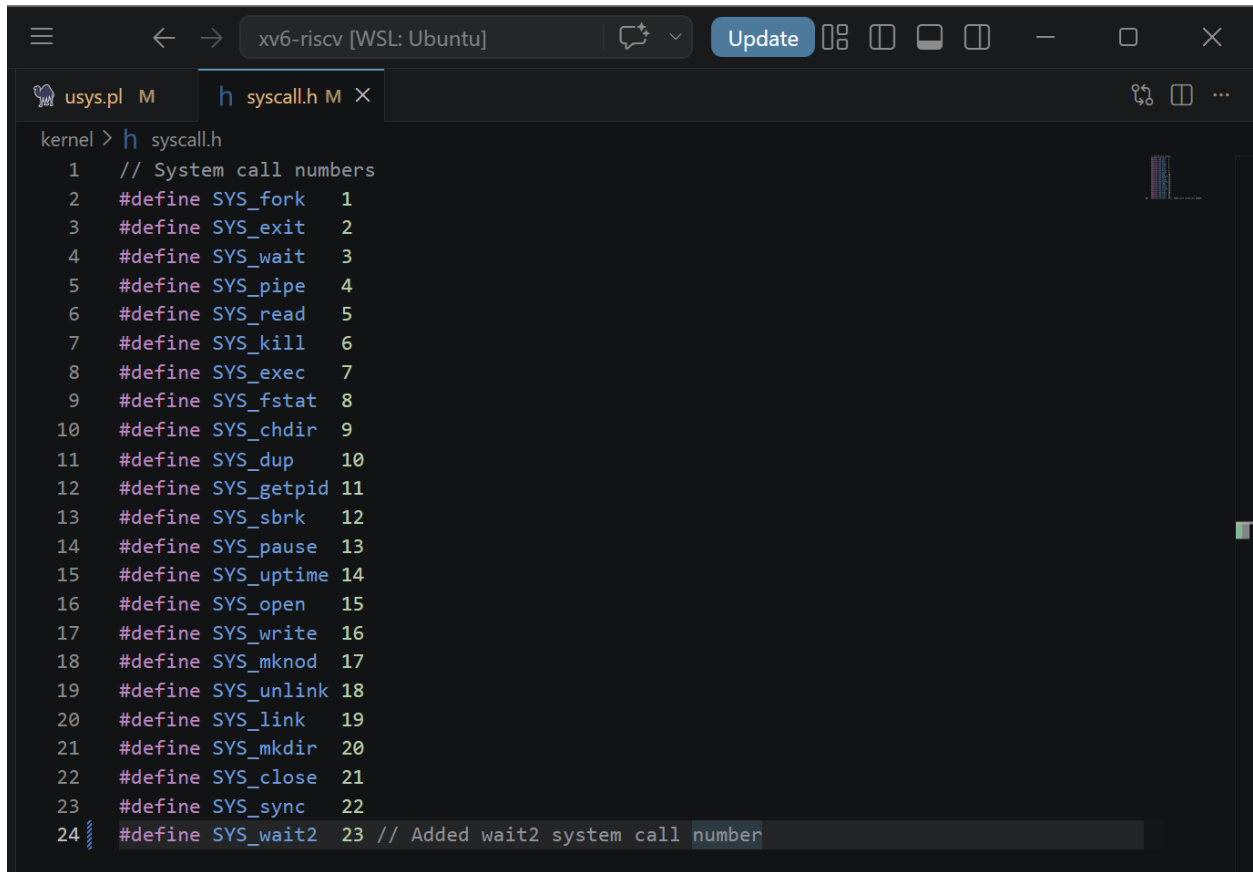
3. user/usys.pl: Added entry("wait2"): This script creates the small assembly code that jumps from a user program into the kernel for each system call.



The screenshot shows a code editor window titled "xv6-riscv [WSL: Ubuntu]". The editor is displaying a file named "usys.pl". The code defines a sub-routine "entry" that takes a system call name as an argument and calls the corresponding kernel function. The system calls listed are: fork, exit, wait, pipe, read, write, close, kill, exec, open, mknod, unlink, fstat, link, mkdir, chdir, dup, getpid, sbrk, pause, uptime, sync, and wait2. The "wait2" entry is commented as "Added wait2, _syscall entry".

```
user > usys.pl
9 sub entry {
22 }
23
24 entry("fork");
25 entry("exit");
26 entry("wait");
27 entry("pipe");
28 entry("read");
29 entry("write");
30 entry("close");
31 entry("kill");
32 entry("exec");
33 entry("open");
34 entry("mknod");
35 entry("unlink");
36 entry("fstat");
37 entry("link");
38 entry("mkdir");
39 entry("chdir");
40 entry("dup");
41 entry("getpid");
42 entry("sbrk");
43 entry("pause");
44 entry("uptime");
45 entry("sync");
46 entry("wait2"); // Added wait2, _syscall entry
```

4. kernel/syscall.h: Gave wait2 its own system call number, 23. Every system call needs a different number so the kernel knows which one the program is asking for.



```
kernel > h syscall.h
1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_fstat 8
10 #define SYS_chdir 9
11 #define SYS_dup 10
12 #define SYS_getpid 11
13 #define SYS_sbrk 12
14 #define SYS_pause 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_sync 22
24 // Added wait2 system call number
```

5. kernel/syscall.c: Added the extern line for sys_wait2 and the line [SYS_wait2] = sys_wait2 in the syscalls table. This is what connects number 23 to the sys_wait2 function, so when a program asks for 23, the kernel knows what to run.

kernel > C syscall.c

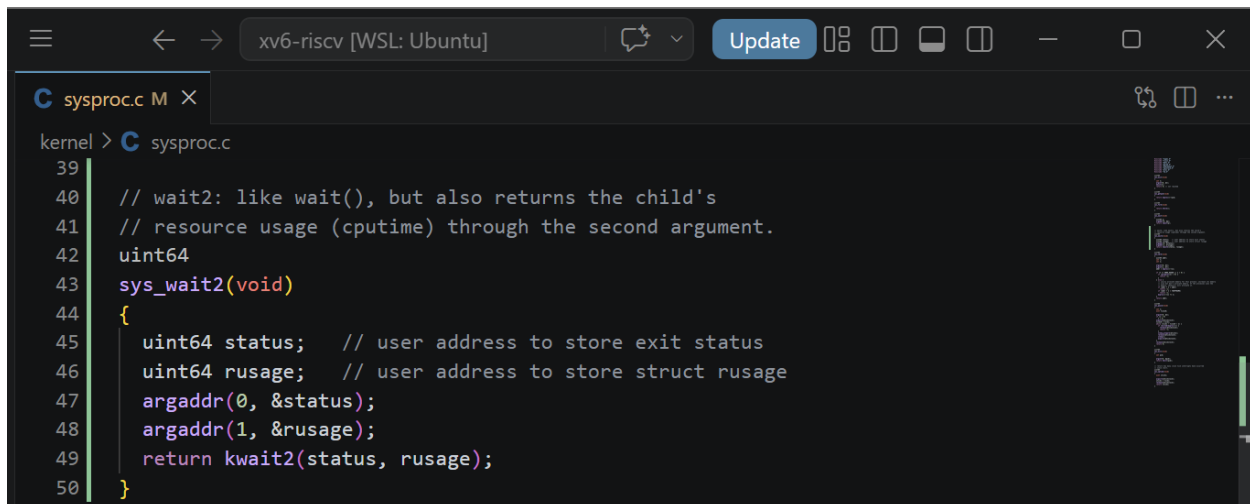
```
//  
81 }  
82  
83 // Prototypes for the functions that handle system calls.  
84 extern uint64 sys_fork(void);  
85 extern uint64 sys_exit(void);  
86 extern uint64 sys_wait(void);  
87 extern uint64 sys_pipe(void);  
88 extern uint64 sys_read(void);  
89 extern uint64 sys_kill(void);  
90 extern uint64 sys_exec(void);  
91 extern uint64 sys_fstat(void);  
92 extern uint64 sys_chdir(void);  
93 extern uint64 sys_dup(void);  
94 extern uint64 sys_getpid(void);  
95 extern uint64 sys_sbrk(void);  
96 extern uint64 sys_pause(void);  
97 extern uint64 sys_uptime(void);  
98 extern uint64 sys_open(void);  
99 extern uint64 sys_write(void);  
100 extern uint64 sys_mknod(void);  
101 extern uint64 sys_unlink(void);  
102 extern uint64 sys_link(void);  
103 extern uint64 sys_mkdir(void);  
104 extern uint64 sys_close(void);  
105 extern uint64 sys_sync(void);  
106 extern uint64 sys_wait2(void); // Added prototype for wait2 system call handler
```

```
kernel > C syscall.c
108 // An array mapping syscall numbers from syscall.h
109 // to the function that handles the system call.
110 static uint64 (*syscalls[])(void) = {
111     // clang-format off
112     [SYS_fork] = sys_fork,
113     [SYS_exit] = sys_exit,
114     [SYS_wait] = sys_wait,
115     [SYS_pipe] = sys_pipe,
116     [SYS_read] = sys_read,
117     [SYS_kill] = sys_kill,
118     [SYS_exec] = sys_exec,
119     [SYS_fstat] = sys_fstat,
120     [SYS_chdir] = sys_chdir,
121     [SYS_dup] = sys_dup,
122     [SYS_getpid] = sys_getpid,
123     [SYS_sbrk] = sys_sbrk,
124     [SYS_pause] = sys_pause,
125     [SYS_uptime] = sys_uptime,
126     [SYS_open] = sys_open,
127     [SYS_write] = sys_write,
128     [SYS_mknod] = sys_mknod,
129     [SYS_unlink] = sys_unlink,
130     [SYS_link] = sys_link,
131     [SYS_mkdir] = sys_mkdir,
132     [SYS_close] = sys_close,
133     [SYS_sync] = sys_sync,
134     [SYS_wait2] = sys_wait2, // Added wait2 system call handler
135     // clang-format on
136 };
```

6. kernel/defs.h: Added the kwait2() prototype so sysproc.c can call a function that lives in proc.c

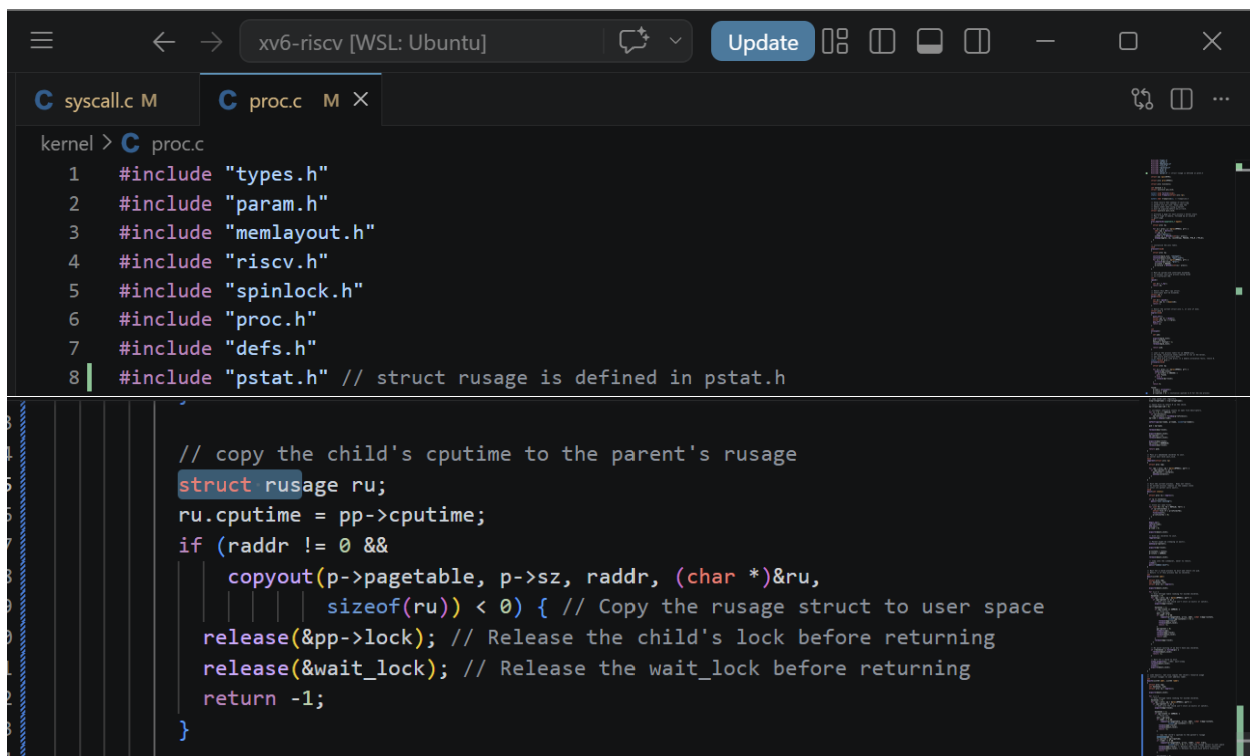
```
kernel > h defs.h
97 void sleep_prepare(void*);
98 void sleep(void);
99 void userinit(void);
100 int kwait(uint64);
101 int kwait2(uint64, uint64); // like wait(), but also returns the child's res
102 void wakeup(void*);
103 void yield(void);
104 int either_copyout(int user_dst, uint64 dst, void *src, uint64 len);
105 int either_copyin(void *dst, int user_src, uint64 src, uint64 len);
106 void procdump(void);
107
```

7. kernel/sysproc.c: sys_wait2() gets the two arguments from the user program (the address where the status should go and the address where the rusage should go) using argaddr(), and then calls kwait2() to do the real work.



```
kernel > C sysproc.c
39
40 // wait2: like wait(), but also returns the child's
41 // resource usage (cputime) through the second argument.
42 uint64
43 sys_wait2(void)
44 {
45     uint64 status; // user address to store exit status
46     uint64 rusage; // user address to store struct rusage
47     argaddr(0, &status);
48     argaddr(1, &rusage);
49     return kwait2(status, rusage);
50 }
```

8. kernel/proc.c: Added #include "pstat.h" so proc.c knows what struct rusage is, and added kwait2(). Kwait2() is a copy of the old function with one new part: when it finds a child that has finished (a ZOMBIE), it puts the child's cputime into a struct rusage and uses copyout() to copy it into the parent's memory. This has to happen before freeproc(), because freeproc() clears out the child's process entry



```
kernel > C proc.c
1 #include "types.h"
2 #include "param.h"
3 #include "memlayout.h"
4 #include "riscv.h"
5 #include "spinlock.h"
6 #include "proc.h"
7 #include "defs.h"
8 #include "pstat.h" // struct rusage is defined in pstat.h

// copy the child's cputime to the parent's rusage
struct rusage ru;
ru.cputime = pp->cputime;
if (raddr != 0 &&
    copyout(p->pagetable, p->sz, raddr, (char *)&ru,
           sizeof(ru)) < 0) { // Copy the rusage struct to user space
    release(&pp->lock); // Release the child's lock before returning
    release(&wait_lock); // Release the wait_lock before returning
    return -1;
}
```

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

Your time.c code should be in the user directory in the pa1-syscalls branch of your xv6 repo.

Remember to add your new command to UPROGS in Makefile.

Show results from running your time command

Basically it works almost the same as time1. The difference is that the parent calls `wait2(&status, &ru)` instead of `wait(0)`. I pass the address of my variables, and then the kernel fills in `ru.cputime`. Then I calculate `%CPU = cputime x 100 / elapsed`. I checked that elapsed is bigger than 0 first, so it doesn't divide by zero for really fast commands. I also include `kernel/pstat.h` so the program knows what struct `rusage` looks like.

```
user > C time.c
1  #include "kernel/types.h"
2  #include "kernel/pstat.h"
3  #include "user/user.h"
4
5  // time: runs a command and reports its elapsed (wallclock) time,
6  // the CPU time it used, and the percentage of the CPU it had.
7  // Usage: time <command> [args...]
8
9  int
10 main(int argc, char **argv)
11 {
12     // Need at least one argument: the command to time
13     if(argc < 2){
14         fprintf(2, "usage: time <command> [args...]\n");
15         exit(1);
16     }
17
18     // Record the clock ticks before starting the command
19     int start = uptime();
20
21     // Create a child process to run the command
22     int pid = fork();
23
24     if(pid < 0){
25         // fork failed
26         fprintf(2, "time: fork failed\n");
27         exit(1);
28     }
29
30     if(pid == 0){
31         // Child: replace this program with the command.
32         // argv+1 is the null-terminated list of its arguments.
33         exec(argv[1], argv+1);
34
35         // exec only returns if it failed
36         fprintf(2, "time: exec %s failed\n", argv[1]);
37         exit(1);
38     }
39
40     // Parent: wait for the child to finish, and get its
41     // resource usage (cputime) from the kernel via wait2()
42     int status;
43     struct rusage ru;
44     wait2(&status, &ru);
45
46     // Record the clock ticks after the command finished
47     int end = uptime();
48     int elapsed = end - start;
49
50     // CPU = share of the elapsed time the child was running.
51     // Avoid dividing by zero if the command finished instantly.
52     int percent = 0;
53     if(elapsed > 0)
54         percent = (ru.cputime * 100) / elapsed;
55
56     printf("elapsed time: %d ticks, cpu time: %d ticks, %d%% CPU\n",
57           elapsed, ru.cputime, percent);
58     exit(0);
59 }
```

```
naomi@LAPTOP-A7FU7E88:~/xv6-riscv$ make qemu
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -ffile-prefix-map=/home/naomi/xv6-riscv/. -march=rv64gc -std=gnu99 -MD -mmodel=medany -f
freestanding -fno-common -nostdlib -fno-builtin-strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcmp -fno-builtin-log -fno-builtin-bzero
-fno-builtin-strchr -fno-builtin-exit -fno-builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-vprintf -fno-builtin-vfprintf -fno
-stack-protector -fno-pie -no-pie -c -o user/time.o user/time.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/time user/time.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/time > user/time.asm
riscv64-unknown-elf-objdump -t user/time | sed '1,/SYMBOL TABLE/d; s/ .* //; /$/d' > user/time.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_matmul user/_mkdir user/_rm user/_sh user/_sleep user/_stressfs user/_time1 u
ser/_time user/_usertests user/_uptime user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_sync
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1155 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk
-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time matmul
Time: 18 ticks
elapsed time: 18 ticks, cpu time: 18 ticks, 100% CPU
$ time matmul & ; time matmul &
$ Time: 35 ticks
elapsed time: 36 ticks, cpu time: 18 ticks, 50% CPU
Time: 34 ticks
elapsed time: 35 ticks, cpu time: 18 ticks, 51% CPU

$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul 3
Time: 53 ticks
elapsed time: 53 ticks, cpu time: 53 ticks, 100% CPU
$ time uptime
up 515 clock ticks
elapsed time: 0 ticks, cpu time: 0 ticks, 0% CPU
$ time
usage: time <command> [args...]
$
```

Results:

Time matmul: 18 ticks elapsed, 18 ticks CPU, 100%. Matmul never waits for anything, so it uses the CPU the whole time.

Time matmul & ; time matmul &: each one took about 35-36 ticks elapsed but still only 18 ticks of CPU, so about 50%. They share one CPU and take turns. The 51% is only because the division rounds down ($18 \times 100 / 35 = 51.4$).

Time sleep 10: 10 ticks elapsed, 0 CPU, 0%. Sleep is just waiting so it doesn't use the CPU.

Time matmul 3: 53 ticks elapsed and 53 CPU, and 100%. It does the work 3 times, so it takes about 3×18

The main thing this shows is the difference between the two kinds of time. Elapsed time changes depending on what else is running, but CPU time stays the same (matmul always used about 18 ticks).

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

Before this lab I didn't really get how a program talks to the kernel. Adding `wait2()` showed me the whole path: the user program calls the function, `usys.pl` makes the code that jumps into the kernel with a number, `syscall.c` uses that number to find the right function, and then the kernel does the work and copies the answer back with `copyout()`. I also learned how `fork()`, `exec()`, and `wait()` work together, and why you need `fork()` before `exec()`. And I also learned the difference between elapsed time and CPU time, which I could actually see when I ran two `matmuls` at the same time.

Some of the difficulties I faced was with the time1, my first version had the parent and the child swapped, so the parent turned into matmul and the elapsed never printed. Looking at the output helped me see which process was doing what and swapping them fixed it.

Another one was some of the kernel changes since in trap.c I forgot the curly braces, so yield() was outside the if and it didn't compile, and I fixed by making sure each function ends with } before the next one starts.

In general, most of the issues I had I solved them by reading the compiler errors carefully, since they tell you the file and line number, and it was easier to fix one error at a time.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

1. It doesn't count the child processes. If the command being timed uses fork() to make its own children, their CPU time isn't included. Wait2() only gives back the cputime of the direct child, and when that child dies its kids are passed to init, so their time is lost.
2. Ticks are too big to be exact since everything is measured in whole ticks, so something short like uptime shows 0 ticks even though it did run.
3. The elapsed time also includes the time for fork() and exec() to set things up, not just the command itself. And if exec() fails, time still prints a result for a command that never ran.